User as Engram: Internalizing Per-User Memory as Local Parametric Edits

Anonymous submission

Abstract

Personal memory requires both mutable user facts and reusable reasoning. Retrieval-augmented generation (RAG) keeps facts external but pays a search or tool hop and serializes retrieved text at query time; per-user LoRA removes that path but stores each user as a global model update. We propose **User as Engram**: write content into hash-addressed memory rows and learn reasoning once in a shared adapter. Against per-user LoRA, the design provides three benefits. At 100 facts it uses 88 KB rather than 14.2 MB ($161\times$ less state); closed-form writes and removable per-user override maps avoid adapter retraining; and it matches direct recall while improving mean indirect reasoning by $5.6\times$, with its largest LOCOMO gains on multi-hop questions. Writes are inspectable and cause about $33{,}000\times$ less held-out loss increase. Against RAG, Engram offers an in-network lookup opportunity at every prefill and decode position, requiring neither external retrieval nor serialized memory tokens. On our controlled split, same-base RAG top-3 performs one search, adds about 29 memory tokens, and reaches 39.0% indirect accuracy versus Engram's 44.5%. A $2.5\times$ larger Qwen-RAG top-3 leads at 34 facts, ties at 100, and falls to 30.0% versus 44.5% at 1,000 facts as retrieval recall degrades. Engram thus targets compact, frequently accessed user state, while RAG remains complementary for provenance and open-domain evidence.

Introduction

Personalization requires both remembering a user's facts and using them to reason. Retrieval-augmented generation (RAG) keeps those facts editable, but recall takes an external systems path: retrieve from a store, serialize the result as text, and pay the resulting context and prefill cost (Lewis et al. 2020; Gao et al. 2023). In an agentic implementation, retrieval also entails generating and executing a tool call. Repeated memory access thus means repeated search or tool round trips. A per-user LoRA (Hu et al. 2022) removes that external path, but turns records into a model-shaped global update: it requires per-user optimization, provides no address for updating or deleting one fact, consumes 14.2 MB at rank 64 even for a small record set, and can perturb unrelated behavior.

We propose *User as Engram*, a middle ground between external retrieval and undifferentiated model weights. We store user content as local edits to the hash-addressed memory table of an Engram-pretrained Transformer (Cheng et al. 2026), while learning reasoning skill once in a shared LoRA. At every token position in prefill or decoding, suffix N-grams can address memory rows whose gated values enter the residual stream in the same forward pass. Memory can therefore supply an in-network signal throughout generation, without semantic search, a tool round trip, or retrieved context tokens. This offers a token-granular interface to frequently accessed user state; it complements rather than replaces RAG's strengths in provenance and open-domain evidence.

Three properties make this interface attractive as a new form of user memory. *Storage:* a 100-fact override map is 88 KB, $161\times$ smaller than a rank-64 per-user LoRA. *Lifecycle:* the representation does not intrinsically require training a per-user adapter—closed-form insertion uses no gradient steps, and an entire user override map can be removed directly; optional row-only refinement can improve recall when facts interact. *Use:* local facts remain available as inputs to one transferable reasoning skill: across three seeds, Engram plus shared skill reaches 41.2% indirect accuracy versus 7.3% for per-user LoRA, and Engram's largest LOCOMO gains occur on multi-hop questions. Figure 4 summarizes these three benefits. A fact activates only the rows selected by its suffixes; all other rows and backbone weights remain bit-identical. Small override maps can be swapped per request or combined additively, separating rapidly changing, private content from slowly learned, population-level skill, as in complementary biological memory systems (Semon 1921; Tonegawa et al. 2015; McClelland, McNaughton, and O'Reilly 1995).

Figure 1 illustrates this separation. A user's doctor or preference may change and require individual deletion; inferring that a cardiologist is relevant to a heart check-up is stable and transferable. A per-user adapter conflates these lifecycles, repeatedly relearning the relation and making individual records difficult to locate or delete. Our design keeps content explicit and reasoning reusable.

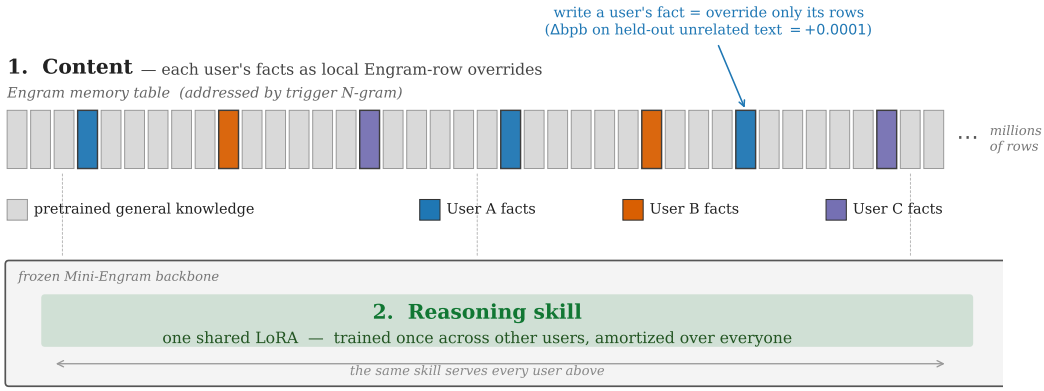

Figure 1: Local Engram content with a shared Transformer and reasoning adapter.

Contributions. We make three contributions. First, we formulate per-user memory as explicit overrides to an Engram model's hash-addressed rows, with closed-form insertion, explicit map deletion, and composable maps. Second, we separate this mutable content from one shared reasoning adapter and verify the interface mechanistically and behaviorally, including indirect and multi-hop use. Third, we provide controlled comparisons with both per-user LoRA and RAG across state, update cost, locality, reasoning, retrieved-memory tokens, and knowledge-base scaling: Engram uses $161\times$ less per-user state than LoRA and no external memory hop, while the $2.5\times$ larger Qwen-RAG baseline falls below it beyond 100 facts (Figures 4, 7, and 8).

Background and Design Requirements

Personal-memory mechanisms span three families. Prompt memories and retrieval stores keep content editable, but consume context and select evidence at every turn (Lewis et al. 2020; Gao et al. 2023). Parameter-efficient tuning moves content into a shared function (Hu et al. 2022). Model editing targets particular associations, yet usually produces weight changes whose locality is empirical (Meng et al. 2022, 2023). Engram pretraining supplies a fourth substrate: explicit key–value rows in the residual stream (Cheng et al. 2026).

We evaluate personal-memory methods along six requirements that are easy to conflate. *Recall* asks whether the exact fact can be produced. *Use* asks whether it can participate in a novel inference. *Locality* asks whether the write leaves unrelated behavior unchanged. *Lifecycle* requires individual update and deletion. *Serving* requires many users to share one backbone. *Access* asks whether each recall needs search, a tool round trip, or extra context. A method that succeeds on recall alone can fail all five remaining requirements; this motivates reporting held-out loss, indirect questions, address-level analyses, per-user storage, and query cost alongside accuracy.

The term *Engram* here denotes the model's pretrained hashed memory table, not an arbitrary cache added after training. Its gates have learned when table values should enter the residual stream. We exploit that learned interface but do not alter its addressing rule or backbone. Our contribution is a write and composition protocol for per-user overrides, plus the separation of those overrides from a shared adapter that learns population-level reasoning.

Why Facts Should Not Be Per-User LoRA

Weights are appropriate for shared abstractions but problematic for mutable, tenant-specific records. We isolate this distinction using Mini-Engram-d20 and the same 20 users. On the canonical seed, a rank-64 LoRA trained on one fact per user raises held-out loss by 1.784 and worsens 17/20 users, whereas addressed Engram rows change loss by only 0.00005 and worsen none. Across three seeds the means are $+1.698$ versus $+0.000052$ ($\sim 33{,}000\times$), and 49/60 versus 0/60 users worsen.

The distinction is architectural, not merely an optimizer failure. A LoRA must move shared projections to raise the desired answer probability, so its update can fire on unrelated inputs. A keyed memory write is inert unless its address is read. Strong instruction-tuned bases can conceal the behavioral damage by already solving a question, but cannot remove the underlying loss increase.

Across Qwen2.5-3B/7B (Qwen Team 2025), Llama-3.1-8B (Grattafiori et al. 2024), and Mistral-7B (Jiang et al. 2023), only 0–20% of users score worse after personalization, and mean indirect accuracy changes by $+0.088$ to $+0.217$; on the base LM, 85% score worse and the mean is -0.133. We fit one LoRA per user: 30 for Qwen2.5-3B and 20 for every other base. Instruction tuning can therefore mask the task-level effect of a global LoRA update, but the update remains nonlocal. Each per-user LoRA is still an alternative global model state, whereas sparse override maps share a backbone.

User as Engram

An Engram layer hashes token-suffix N-grams to memory rows, gates their values, and adds them to the residual stream. To write a fact, we put its answer where its trigger is read: deterministic hashes map the trigger's suffix to a sparse address set R_f (about 16 rows for $N = 3$, $K = 8$). The backbone and every row outside R_f remain fixed. We evaluate three ways to choose the values written to R_f. A closed-form pseudo-inverse write (*UNEMBED_P*) uses the Engram value projection's pseudo-inverse to align the row contribution with the

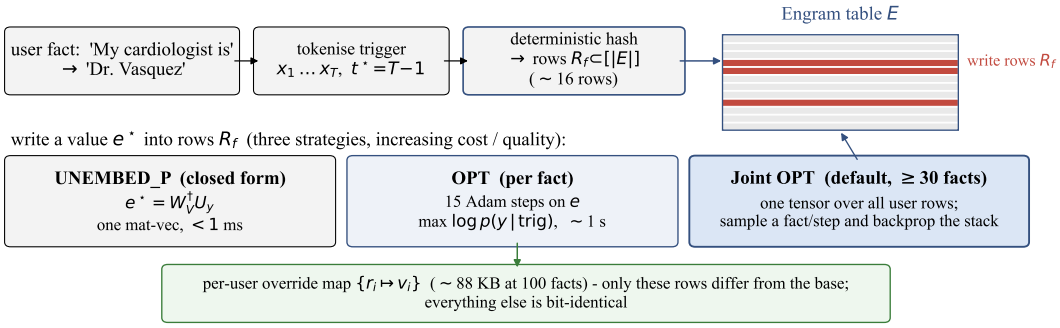

Figure 2: Surgical insertion into deterministically addressed Engram rows.

answer token's unembedding direction and requires no gradient steps. Per-fact gradient optimization (*OPT*) starts from that value and minimizes answer loss by updating only R_f for each arriving fact. Joint optimization (*Joint OPT*) instead samples across a user's facts while fitting the union of their addressed rows, so earlier and co-active facts remain compatible. All three strategies have the same editable parameter set; they differ only in write compute and recall. A 100-fact override map occupies about 88 KB. Figure 2 summarizes the write process.

The edit is a glass box. Figure 3 contrasts its footprint with the matched LoRA. For an inserted fact we can enumerate the trigger hashes, verify that only R_f changed, inspect the learned gates, measure the residual contribution at each layer, and observe the answer token rise in the logit lens. In the matched diagnostic, Engram's residual delta is exactly zero at every non-trigger position and every pre-Engram layer; the LoRA delta is nonzero at every position and layer. On the unrelated "capital of France" control, its mean L_2 residual-stream change across positions and layers is 107. Locality therefore follows from the data structure rather than from a regularizer.

Keys, correctness, and deletion. Personal questions often reproduce an entity or relation phrase even when surrounding wording changes. Multiple suffix orders and hashes provide redundant triggers; inserting paraphrased question forms expands coverage. This is not semantic search: a query with no overlapping trigger may miss. Because addresses are deterministic, we know the complete editable row set before optimization. Deleting a user's map removes all of its overrides without reversing a gradient update. An individual fact can be removed directly when it exclusively owns its rows; if collisions or joint optimization make rows shared, those rows must instead be reconstructed from the remaining facts. Closed-form writes need no training, while optional joint reoptimization can repair interference when facts share rows.

Mechanistic checks. We validate locality four ways: byte comparison leaves the backbone and unaddressed rows identical; non-trigger positions have zero residual delta; the trigger gate rises from 0.02 to 0.99; and the residual change aligns with the value path at cosine 0.999. The same write placed early drops recall from near-perfect to about one quarter, confirming that late placement matters.

The comparison LoRA is optimized to the same recall target. Its residual delta appears at all positions and layers, including an unrelated "capital of France" control, because low-rank does not mean locally activated. Norm or KL regularization might reduce that footprint, but would not make the update address-conditioned or identify exactly which inputs can activate it. For Engram, the zero-delta region follows directly from the lookup key.

Each user owns an override map $\Delta_u : h \mapsto v$. At inference, lookups first consult that map and otherwise fall back to the shared table. Maps with disjoint keys commute, so personal, team, and organizational memories can be stacked without merging dense weight deltas. Hash collisions are explicit and can be detected. In the current implementation the last write wins; maps whose triggers overlap should therefore remain separate.

Experiments

No suitable public pretrained Mini-Engram checkpoint was available when we ran these experiments. We therefore integrate Engram lookup layers into Karpathy's nanochat GPT-2-style backbone (Karpathy 2026; Radford et al. 2019) and pretrain all four Mini-Engram checkpoints ourselves from scratch with a nanochat-style recipe. We evaluate them against per-user LoRA, in-context facts, and retrieval, matching backbone, tokenizer, generation, and facts unless noted. Metrics cover direct and paraphrase recall, held-out LM loss, per-user storage, and indirect questions that transform stored facts.

Controlled bases vary depth and width but share the lookup. We report accuracy and held-out next-token loss because strong models can mask contamination once answers saturate. Storage excludes the backbone and shared adapter, which are amortized across tenants.

Synthetic per-user schemas isolate fact count and separate exact recall from held-out transformations. LOCOMO tests end-to-end conversational memory against retrieval; its question-level evidence pointers let both methods use the same gold sentence. Direct questions request stored answers, whereas indirect questions apply held-out relations and cannot be solved by echoing triggers. We repeat key contamination and layering results over three seeds. Writes are offline; query cost includes lookup, search, and context, and storage means serialized trainable state rather than duplicated

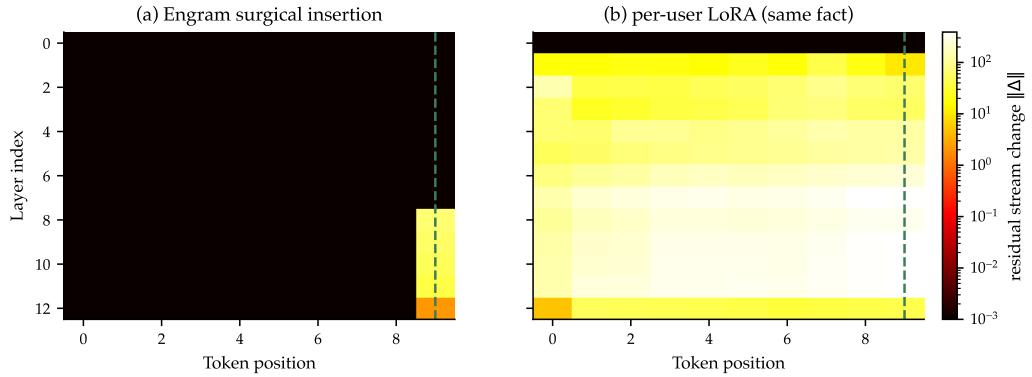

Figure 3: Residual changes from matched Engram and per-user LoRA edits.

Table 1: Mini-Engram backbones pretrained by us from scratch. All share the 51.2M-parameter memory table.

	d8 v2	d12 v2	d12@1280	d20@1536
depth × width	8 × 512	12 × 768	12 × 1280	20 × 1536
Engram layers	2, 5	2, 7	2, 7	2, 11
total parameters	178M	339M	625M	1.22B
ClimbMix tokens	0.50B	1.32B	3.34B	7.40B
validation bpb	.912	.827	.770	.730

checkpoints.

Protocol and comparison controls

All Mini-Engrams use one lookup rule and a 50K×256 (51.2M-parameter) table; Table 1 lists their scales. For pretraining, we use bf16, nanochat's ClimbMix-400B pipeline, and its budget of 12 tokens per scaling parameter (Karpathy 2026). Base, XL, and XXL corpora contain 100 USER + 100 ORG facts, 1,000 of each, and 3,132 trigger templates, respectively. At 100 facts, Joint OPT uses 2,000 Adam steps at rate 0.5. The three-seed test matches rank-64 LoRA (1,500 steps, 5×10^{-4}, $\alpha = 128$) against Joint OPT (1,500 steps, rate 0.5).

Metrics. We rank the gold first BPE token at the trigger: top-1 is rank 0 and top-5 is rank below 5. Mini-Engram's `indirect_any` lowercases and strips a 16-token greedy continuation and tests gold substring containment; Qwen uses its decoded-text matcher, which also normalizes terminal punctuation and accepts an equal first word. LOCOMO token-F1 normalizes case and punctuation, then macro-averages conversation means. Layered scores average per-user rates. Contamination is held-out ClimbMix Δbpb (262K tokens for S0; 524,288 for the retained additional-seed run). RAG memory tokens are prompt tokens beyond the no-memory prompt; serving reports requests/wall time and non-interpolated latency order statistics.

The shared rank-16 adapter trains on 10 users and tests on 20 disjoint users; contamination and high-density comparisons use rank-64 per-user Q/K/V LoRA. Unless stated otherwise, each checkpoint, condition, or curve point is one execution, and user-specific methods fit once per evaluated user. We train each backbone once; run each LOCOMO

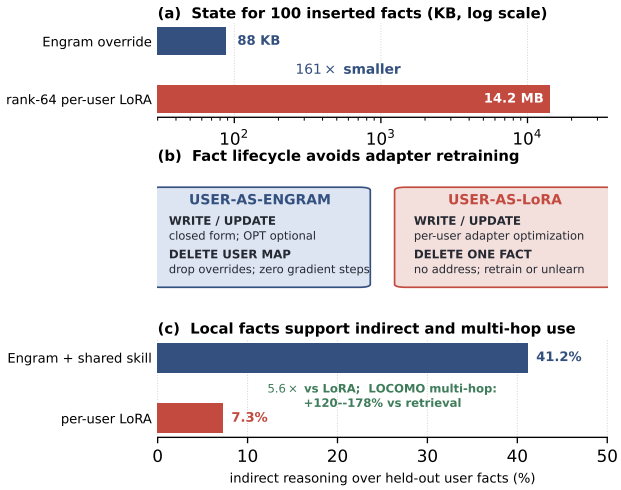

Figure 4: Engram-row benefits over per-user LoRA: **(a)** storage, **(b)** lifecycle, and **(c)** downstream use.

model/category and the 63-chain diagnostic once; use one seeded RAG sweep per answering model; and use one 600-request serving run per configuration. Storage and byte counts are deterministic. The confidence interval bootstraps paired outputs from the three seed runs.

Corpus-generation seeds are 31337, 20260505, and 20260506; shared-LoRA training uses seed 42 and RAG distractors use seed 0. Runs used one NVIDIA RTX PRO 6000 Blackwell GPU with bf16.

No-memory measures prior knowledge. For indirect reasoning, relations are learned from other users and test content enters only through the evaluated memory, requiring unseen-content access and transferable skill. Unrelated held-out ClimbMix makes Δbpb measure collateral change, not fact recall.

Recall, storage, and lifecycle. A single edit is within eight points of the in-context ceiling. On Mini-Engram-d12, joint optimization handles interacting addressed rows at high density, with top-5 recall above 90% to roughly 300 facts/user. At 100 facts, its sparse override occupies 88 KB versus 14.2 MB

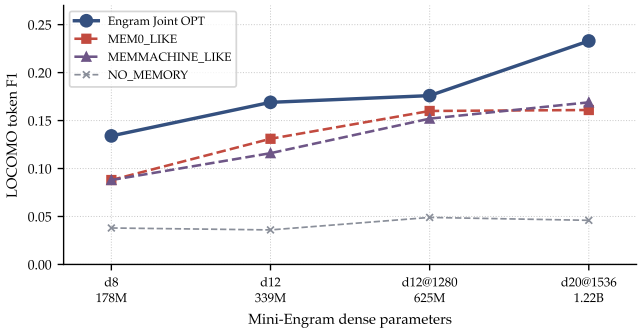

Figure 5: LOCOMO token-F1 across answering models.

for rank-64 LoRA. Unlike a model-shaped adapter, each row remains an explicit item that can be replaced or removed; UNEMBED_P provides a no-training write path, while OPT and Joint OPT trade write compute for recall. Figure 4 summarizes these storage, lifecycle, and downstream-use distinctions.

Memory-system and LOCOMO comparisons. Nearest-neighbor retrieval is strongest when queries repeat stored triggers. Because facts are already structured, our MEM0 and MemMachine baselines use only their retrieval recipes; extraction and consolidation are out of scope (Chhikara et al. 2025; Wang et al. 2026). With all-MiniLM-L6-v2 (Wang et al. 2020; Reimers and Gurevych 2019), multi-trigger Engram gains 22 paraphrase points over the best retrieval baseline without context tokens.

Single-hop LOCOMO uses the first 80 non-adversarial questions from each of 10 conversations (∼800) (Maharana et al. 2024). With a fixed Mini-Engram-d12 answering LM, retrieval stores gold evidence turns and Engram stores gold Q/A pairs: a controlled best-case comparison, not an end-to-end system. Joint OPT leads on single-hop, multi-hop, and reasoning across scales but trails on open-domain questions, where local facts cannot replace broad knowledge (Table 2; Figure 5). LongMemEval and PersonaMem-v2 provide complementary long-term and personalized memory evaluations (Wu et al. 2025; Jiang et al. 2025).

Joint OPT's largest gains are multi-hop: token-F1 improves 120–178% over the better retrieval baseline, versus 10–58% on single-hop and reasoning. Open-domain performance falls 51–57%, favoring addressed personal memory plus retrieval for broad, changing knowledge.

Together with Figure 4's LoRA comparison, these results show that Engram facts can serve as operands for held-out transformations and multi-hop questions. Rows do not supply the algorithm; one shared skill consumes local facts instead of relearning content and skill in every user's adapter.

The paraphrase result separates address coverage from answer quality. Retrieval is nearly perfect on repeated prefixes; more trigger variants improve Engram paraphrases without prompt tokens. Neither implies universal semantic generalization: a stored key must activate.

Multi-hop use. Across 63 two-fact chains, fact recall is 99.2% and direct second-fact matching is 90.6%, but true

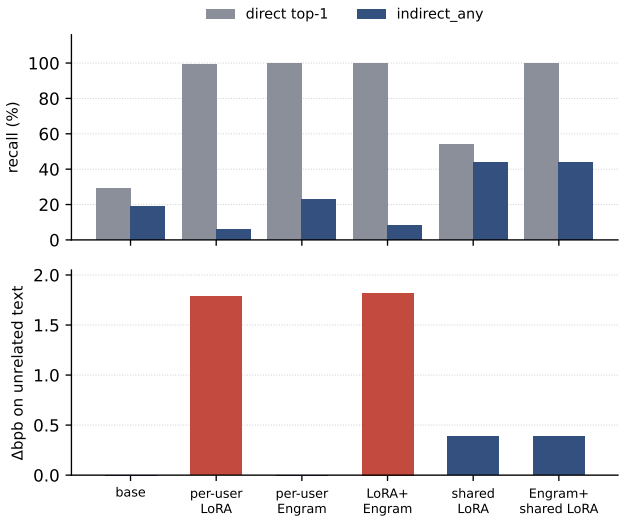

Figure 6: Direct and indirect recall and held-out Δbpb across six conditions on Mini-Engram-d20 (seed S0, $n = 20$).

chaining falls to 12.9%: the store matches but does not compose. We therefore learn relations in a shared adapter on held-out users while fitting only test-user rows. The resulting layered system reaches 41.2% indirect accuracy across seeds, $5.6\times$ the per-user-LoRA baseline; the LOCOMO category results above independently show the largest gains on multi-hop questions.

Interpreting the comparisons. Across Table 3's three seeds, matched recall isolates storage location, density tests coexistence, and layering tests whether locality sacrifices use. Recall need not cause global contamination, and reasoning can be shared; LOCOMO shows that addressed memory does not replace open-domain retrieval.

The $33{,}000\times$ loss ratio is a model-, fact-, and optimization-dependent stress test, not a constant. The robust distinction is qualitative: Engram can be inactive off-address, whereas LoRA changes the shared mapping. Likewise, 88 KB is the chosen 100-fact configuration, not a lower bound; width, precision, triggers, and collisions change it.

Negative results and boundary cases. UNEMBED_P is fast but underperforms when pretrained values misalign with the answer. Independent fitting degrades under trigger collisions, and joint fitting repairs only part of the high-density loss. Free continuation is harder than short-span matching; unseen paraphrase triggers can fail. An arbitrary reasoning LoRA also fails unless it learns the same content interface, motivating layered training.

Shared Skill, Local Content

We train the rank-16 shared LoRA on 10 users with each indirect question's needed facts in context, then test 20 disjoint users whose facts arrive only through Engram rows. Six conditions cross content placement (none, per-user LoRA, or Engram) with shared skill while holding direct recall nearly

Table 2: LOCOMO token-F1 by category and model. Joint OPT is compared with the stronger retrieval baseline in each cell.

category	answering LM	MEM0	MemMachine	**Engram Joint OPT**	lead vs. best retr.
single-hop	d12 v2	0.131	0.116	**0.169**	+29%
	d12@1280	0.160	0.152	**0.176**	+10%
	d20	0.161	0.169	**0.233**	+38%
multi-hop	d12 v2	0.092	0.076	**0.243**	**+165%**
	d12@1280	0.115	0.104	**0.253**	**+120%**
	d20	0.102	0.108	**0.299**	**+178%**
reasoning	d12 v2	0.117	0.113	**0.183**	+56%
	d12@1280	0.123	0.136	**0.203**	+49%
	d20	0.135	0.168	**0.266**	+58%
open-domain	d12 v2	**0.247**	0.209	0.122	**−51%**
	d12@1280	**0.341**	0.327	0.146	**−57%**
	d20	0.314	**0.338**	0.159	**−53%**

Table 3: Three-seed robustness on Mini-Engram-d20 ($n = 20$ users per seed).

Quantity	S0	S1	S2	Mean (range)
per-user LoRA Δbpb	1.784	1.754	1.557	1.698 (1.56–1.78)
per-user Engram Δbpb	.000052	.000049	.000054	.000052
layered total Δbpb	.386	.388	.379	.385 (.38–.39)
layered indirect_any	44.5%	44.2%	34.8%	41.2% (34.8–44.5)
per-user LoRA indirect_any	6.0%	8.8%	7.2%	7.3% (6.0–8.8)
worse than base (layered / LoRA)	0/20 / 17/20	0/20 / 16/20	0/20 / 16/20	0/60 / 49/60

constant. Across three seeds, layered Engram reaches 100% versus 98.8% direct recall and 41.2% versus 7.3% indirect accuracy for per-user LoRA.

The layered system pays $+0.385$ Δbpb once for the shared adapter; each user's rows add only ≈ 0.00005, versus $+1.698$ for a separate per-user LoRA. The adapter learns a reusable transformation while local rows supply user-specific arguments (Figure 6).

The layered-minus-LoRA indirect gain has paired-bootstrap 95% CI $[+31, +37]$ percentage points.

The factorial design attributes the gain: Engram without shared skill tests recall, shared skill without content tests leakage, and naive LoRA+Engram tests arbitrary stacking. Only intended layering gives local content, transferable reasoning, and low held-out loss; train and test users are disjoint.

Cross-schema transfer. The headline comparison uses disjoint users within one schema family. To test beyond those templates, we train on first-person personal facts and evaluate on third-person medical records. Layered indirect accuracy falls from 44% to 31% (30% relative), while per-user LoRA falls from 6% to 4%; the advantage persists ($7.4\times$ to $7.6\times$). Locality is unchanged: Δbpb stays $+0.386$ because personal rows activate only at addressed triggers.

Figure 7 compares the memory-access paths on the 34-fact split. Layered Engram reaches 44.5% indirect accuracy with no retrieved fact text or external memory call. RAG top-3 on the same answering model performs one search, serializes about 29 additional memory tokens, and reaches 39.0%; including all facts raises the memory block to 287 tokens and lowers accuracy to 35.3%. Combining RAG top-3 with the shared reasoning adapter instead reaches 54.3%, showing that retrieval and reusable skill are complementary. The $2.5\times$ larger Qwen2.5-3B-Instruct RAG baseline (Qwen Team 2025) reaches 51.8% with top-3 retrieval and 35 added memory tokens.

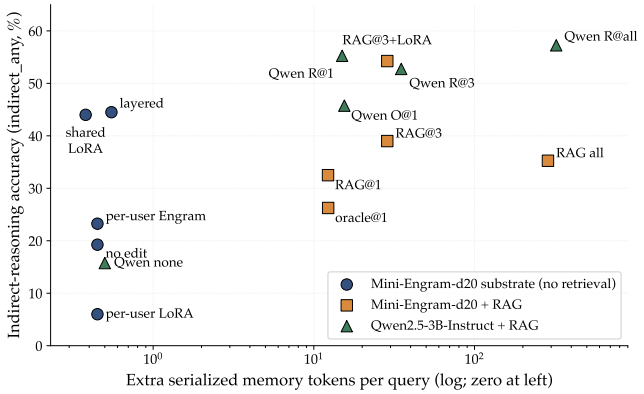

Figure 7: Engram versus RAG: indirect accuracy against extra serialized memory tokens per query; RAG points each perform one search.

The distinction is access granularity. RAG searches once and serializes facts; agentic or sequential recalls add external hops. Engram instead computes suffix addresses at every prefill and decode position and injects values in the forward pass, supplying a token-granular channel without retrieved context. RAG remains the better complement for changing

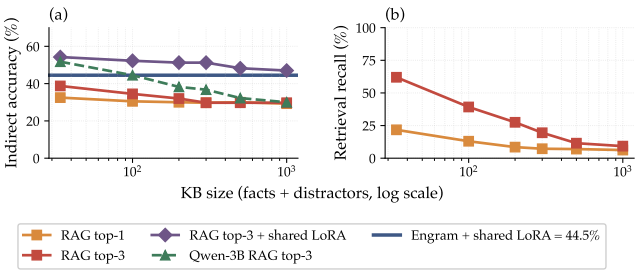

Figure 8: Engram versus RAG as the knowledge base grows: **(a)** indirect accuracy and **(b)** shared retriever recall. Qwen-3B uses a $2.5\times$ larger answering model.

corpora, provenance, and open-domain evidence.

To isolate knowledge-base size, we add held-out schema-family distractors to each user's 34 facts, reaching $N \in \{34, 100, 200, 300, 500, 1000\}$ for the same 20 users and probes. Mini-Engram RAG shares Engram's answering backbone; Qwen-RAG uses the larger instruction-tuned model. All RAG conditions share one retriever and place the top-k facts in context. Top-3 recall falls from 62.0% to 9.3%; Qwen-RAG top-3 falls from 51.8% to 30.0%, tying Engram's constant 44.5% at 100 facts and trailing thereafter. Same-base RAG falls from 38.8% to 29.8%; RAG plus the shared adapter remains slightly ahead at 1,000 facts (47.0%), favoring a hybrid when provenance is needed (Figure 8).

Multi-Tenant Serving

A request resolves a sparse user table, overrides affected base rows for one forward pass, then restores them. Each token suffix can fetch a gated value into the residual stream without a retrieval service or memory text to prefill. User switches change neither dense weights nor the shared adapter. Disjoint personal, team, and organizational scopes can layer; overlapping triggers are last-write-wins and should remain separate. Unmeasured batched serving would require one gather per Engram layer.

At one million users, 88 KB each is roughly 88–100 GB, plus one 11.8 MB shared adapter; 14.2 MB per-user LoRAs require about 14.2 TB. Excluding allocator and index overheads, sparse content scales with facts while LoRA scales with model dimensions.

S-LoRA and Punica (Sheng et al. 2024; Chen et al. 2024) batch dense adapters, but each tenant retains a model-shaped delta; overrides are indexed reads in the existing layer. Production still requires caching, authorization, encryption, collision telemetry, and rate limits; we test composition and scaling, not security.

The prototype uses sequential row swaps, not a batched kernel. On one idle Blackwell GPU, d12@1280 processed 232 requests/s with 30 users and 50 facts/user (4.2 ms p50), and 226 requests/s with 100 users and 100 facts/user (4.4 ms p50). This is not a controlled user-count scaling study because fact count also changes; absolute latency depends on cache residency, hardware, and implementation.

Discussion, Limitations, and Related Work

At 1,000 facts Joint OPT reaches 35% top-1; with $<1\%$ slot occupancy, this suggests that forward-pass interference among co-active rows, rather than hash collisions, is the primary bottleneck. Paraphrase coverage depends on triggers, and optimization is slower than appending a document.

Our work connects parameter-efficient personalization and per-user adapters (Houlsby et al. 2019; Hu et al. 2022; Zhang et al. 2024; Su et al. 2025; Tan, Liu, and Jiang 2024; Zhuang et al. 2024), retrieval-augmented generation (Lewis et al. 2020; Gao et al. 2023; Borgeaud et al. 2022), model editing (Meng et al. 2022, 2023; Mitchell et al. 2022a,b; Wang et al. 2024), and external or semi-parametric memory (Lample et al. 2019; Packer et al. 2023; Cheng et al. 2026). Retrieval and external memory provide non-parametric access; adapters and model editing change parameters with different scopes and locality. User as Engram lies between them: records remain explicit and mutable, but the network consumes their values without retrieval calls or context tokens. Addressed rows hold mutable facts, while shared weights hold reusable reasoning, yielding locality, high-frequency access, and multi-tenancy rather than dense per-user models. The dense-adapter interference we measure is related to catastrophic forgetting in continual learning (McCloskey and Cohen 1989; Kirkpatrick et al. 2017).

Validity and societal considerations. Controlled models expose changed rows and small loss deltas, strengthening causal claims but limiting frontier-scale conclusions. LOCOMO mixes memory and base knowledge, so we report category reversals. Storage and latency constants are implementation-dependent, though scaling forms are robust. Addressability aids erasure but can expose records; exfiltration, access control, output monitoring, and shared-skill fairness remain untested. Generative AI assisted language editing, code, and formatting; the authors verified all outputs and remain responsible for the manuscript.

What should improve next. Three recipe changes target these limits: multi-fact training, random per-user hash offsets for unseen rows, and "recall, then reason" examples (Sun et al. 2023). Preliminary multi-fact finetuning reached the same recall faster without raising its ceiling; teacher traces lowered accuracy when training and evaluation formats differed. Frontier-scale pretraining is the strongest next comparison. We expect addressed memory for frequent records, retrieval for long-tail evidence and provenance, and shared weights for stable transformations.

Conclusion

User as Engram separates per-user content from shared reasoning by writing facts to hash-addressed rows and learning one adapter. Across experiments, it matches direct recall, improves indirect reasoning $5.6\times$ over per-user LoRA, and uses $161\times$ less state at 100 facts. Closed-form writes require no gradient steps, while optional row-only optimization improves recall. Its token-granular lookup removes external memory hops and is unaffected by the added distractors in our controlled retrieval-scaling comparison.

References

Borgeaud, S.; Mensch, A.; Hoffmann, J.; et al. 2022. Improving Language Models by Retrieving from Trillions of Tokens. In *Proceedings of the International Conference on Machine Learning*.

Chen, L.; Ye, Z.; Wu, Y.; et al. 2024. Punica: Multi-Tenant LoRA Serving. In *Proceedings of MLSys*.

Cheng, X.; Zeng, W.; Dai, D.; Chen, Q.; Wang, B.; Xie, Z.; Huang, K.; Yu, X.; Hao, Z.; Li, Y.; Zhang, H.; Zhang, H.; Zhao, D.; and Liang, W. 2026. Conditional Memory via Scalable Lookup: A New Axis of Sparsity for Large Language Models. arXiv:2601.07372.

Chhikara, P.; Khant, D.; Aryan, S.; Singh, T.; and Yadav, D. 2025. Mem0: Building Production-Ready AI Agents with Scalable Long-Term Memory. arXiv:2504.19413.

Gao, Y.; et al. 2023. Retrieval-Augmented Generation for Large Language Models: A Survey. arXiv:2312.10997.

Grattafiori, A.; Dubey, A.; Jauhri, A.; et al. 2024. The Llama 3 Herd of Models. arXiv:2407.21783.

Houlsby, N.; et al. 2019. Parameter-Efficient Transfer Learning for NLP. In *Proceedings of the International Conference on Machine Learning*.

Hu, E.; Shen, Y.; Wallis, P.; Allen-Zhu, Z.; Li, Y.; Wang, S.; Wang, L.; and Chen, W. 2022. LoRA: Low-Rank Adaptation of Large Language Models. In *International Conference on Learning Representations*.

Jiang, A. Q.; Sablayrolles, A.; Mensch, A.; et al. 2023. Mistral 7B. arXiv:2310.06825.

Jiang, B.; Yuan, Y.; Shen, M.; et al. 2025. PersonaMem-v2: Towards Personalized Intelligence via Learning Implicit User Personas and Agentic Memory. arXiv:2512.06688.

Karpathy, A. 2026. nanochat: An Experimental Training Harness for LLMs.

Kirkpatrick, J.; Pascanu, R.; Rabinowitz, N.; et al. 2017. Overcoming Catastrophic Forgetting in Neural Networks. *Proceedings of the National Academy of Sciences*, 114(13): 3521–3526.

Lample, G.; Sablayrolles, A.; Ranzato, M.; Denoyer, L.; and Jégou, H. 2019. Large Memory Layers with Product Keys. In *Advances in Neural Information Processing Systems*.

Lewis, P.; Perez, E.; Piktus, A.; Petroni, F.; Karpukhin, V.; Goyal, N.; Küttler, H.; Lewis, M.; Yih, W.; Rocktäschel, T.; Riedel, S.; and Kiela, D. 2020. Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks. In *Advances in Neural Information Processing Systems*.

Maharana, A.; et al. 2024. LOCOMO: Evaluating Very Long-Term Conversational Memory of LLM Agents. In *Proceedings of the Annual Meeting of the Association for Computational Linguistics*.

McClelland, J. L.; McNaughton, B. L.; and O'Reilly, R. C. 1995. Why There Are Complementary Learning Systems in the Hippocampus and Neocortex. *Psychological Review*, 102(3): 419–457.

McCloskey, M.; and Cohen, N. J. 1989. Catastrophic Interference in Connectionist Networks: The Sequential Learning Problem. In *Psychology of Learning and Motivation*, volume 24, 109–165. Academic Press.

Meng, K.; Bau, D.; Andonian, A.; and Belinkov, Y. 2022. Locating and Editing Factual Associations in GPT (ROME). In *Advances in Neural Information Processing Systems*.

Meng, K.; et al. 2023. MEMIT: Mass-Editing Memory in a Transformer. In *International Conference on Learning Representations*.

Mitchell, E.; Lin, C.; Bosselut, A.; Finn, C.; and Manning, C. D. 2022a. Fast Model Editing at Scale. In *International Conference on Learning Representations*.

Mitchell, E.; Lin, C.; Bosselut, A.; Manning, C. D.; and Finn, C. 2022b. Memory-Based Model Editing at Scale. In *Proceedings of the International Conference on Machine Learning*.

Packer, C.; Wooders, S.; Lin, K.; et al. 2023. MemGPT: Towards LLMs as Operating Systems. arXiv:2310.08560.

Qwen Team. 2025. Qwen2.5 Technical Report. arXiv:2412.15115.

Radford, A.; Wu, J.; Child, R.; Luan, D.; Amodei, D.; and Sutskever, I. 2019. Language Models Are Unsupervised Multitask Learners (GPT-2). Technical report, OpenAI.

Reimers, N.; and Gurevych, I. 2019. Sentence-BERT: Sentence Embeddings Using Siamese BERT-Networks. In *Proceedings of EMNLP-IJCNLP*.

Semon, R. 1921. *The Mneme*. London: George Allen & Unwin.

Sheng, Y.; Cao, S.; Li, D.; et al. 2024. S-LoRA: Serving Thousands of Concurrent LoRA Adapters. arXiv:2311.03285.

Su, W.; et al. 2025. Parametric Retrieval-Augmented Generation (PRAG). arXiv:2501.15915.

Sun, Z.; et al. 2023. Recitation-Augmented Language Models. In *International Conference on Learning Representations*.

Tan, Z.; Liu, Q.; and Jiang, M. 2024. Democratizing Large Language Models via Personalized Parameter-Efficient Fine-Tuning (OPPU). In *Proceedings of EMNLP*.

Tonegawa, S.; Liu, X.; Ramirez, S.; and Redondo, R. 2015. Memory Engram Cells Have Come of Age. *Neuron*, 87(5): 918–931.

Wang, P.; Li, Z.; Zhang, N.; et al. 2024. WISE: Rethinking the Knowledge Memory for Lifelong Model Editing of Large Language Models. In *Advances in Neural Information Processing Systems*.

Wang, S.; Yu, E.; Love, O.; Zhang, T.; Wong, T.; Scargall, S.; and Fan, C. 2026. MemMachine: A Ground-Truth-Preserving Memory System for Personalized AI Agents. arXiv:2604.04853.

Wang, W.; Wei, F.; Dong, L.; Bao, H.; Yang, N.; and Zhou, M. 2020. MiniLM: Deep Self-Attention Distillation for Task-Agnostic Compression of Pre-Trained Transformers. In *Advances in Neural Information Processing Systems*.

Wu, D.; Wang, H.; Yu, W.; Zhang, Y.; Chang, K.-W.; and Yu, D. 2025. LongMemEval: Benchmarking Chat Assistants on Long-Term Interactive Memory. In *International Conference on Learning Representations*.

Zhang, Z.; Rossi, R. A.; Kveton, B.; et al. 2024. Personalization of Large Language Models: A Survey. arXiv:2411.00027.

Zhuang, Y.; et al. 2024. HYDRA: Per-User Adapters for Personalised LLMs.